Lab Session 5

Introduction and Working Of SOLID Design Principles in Software Construction

Objective

To understand and apply SOLID design principles using real-world software scenarios. Students will learn how proper design decisions improve maintainability, scalability, and flexibility in software systems.

Introduction

In modern software development, building a system that only works is not enough, it must also be **easy to maintain, extend, test, and scale**. Real-world software systems such as hospital systems, banking apps, e-commerce platforms, and university portals continuously evolve as user needs, business rules, and technologies change. New features are added, policies are updated, and user loads increase.

If a system is poorly designed, even small changes can introduce bugs, break existing functionality, or require large portions of the system to be rewritten. This leads to higher development costs, longer delivery times, and frustrated developers and users.

This is where **design principles** become important. Design principles provide proven guidelines for structuring software so that it remains flexible and robust over time. They help developers organize code in a way that reduces complexity, improves readability, and minimizes dependencies between components.

Using proper design principles helps to:

- Reduce tight coupling between modules
- Improve code reusability
- Make debugging and testing easier
- Support future enhancements without major rewrites
- Increase system stability and lifespan

One of the most widely accepted sets of design guidelines in object-oriented programming is the **SOLID principles**. These principles focus on writing clean, modular, and adaptable code. Instead of solving only today's problem, SOLID helps developers design systems that can handle future changes gracefully.

In this lab, we explore SOLID principles through real-world scenarios to understand not only *what* these principles are, but also *why* they are essential in professional software development.

1. Single Responsibility Principle (SRP)

Definition

A class should have only one responsibility and one reason to change.

Scenario: Hospital Management System

A hospital develops a system to manage patient records. Initially, the system is small. A developer creates a single class called PatientManager that:

- Stores patient details
- Calculates billing
- Sends appointment reminders via email

At first, everything works fine.

Problem Over Time

After a few months:

1. **Billing rules change** due to new insurance policies.
   → Billing logic must be updated.
2. Hospital switches from **email to SMS reminders**.
   → Notification logic must be changed.
3. Government regulations require **data format changes**.
   → Data storage logic must be modified.

All these changes require editing the same class.

This causes:

- High risk of bugs
- One fix breaking another feature
- Difficult testing
- Developer confusion

One class is doing too many jobs.

❌ Code (SRP Violation)

```
class PatientManager {
    void savePatient(){}
    void generateBill(){}
    void sendReminder(){}
}
```

✅ SOLUTION (Apply SRP)

Split Responsibilities

```java
class PatientRepository {
    void savePatient(){}
}

class BillingService {
    void generateBill(){}
}

class NotificationService {
    void sendReminder(){}
}
```

Explanation

Now:

- Billing change affects only BillingService
- Notification change affects only NotificationService
- Patient data remains safe

Teams can work independently.
Testing becomes easier.
System becomes modular.

2. Open/Closed Principle (OCP)

Definition

Classes should be open for extension but closed for modification.

Scenario: Online Shopping Platform

An e-commerce company runs seasonal campaigns. Initially, they offer:

- Student discounts
- Festival discounts

A developer writes a simple discount calculator.

Problem Over Time

The company grows. Marketing introduces:

- Black Friday discounts
- VIP customer discounts
- Loyalty rewards
- Flash sale discounts

Each time, developers modify the discount class.

Soon:

- Old discount logic breaks
- Bugs appear
- Testing takes longer
- Developers fear touching the class

The class becomes fragile.

❌ Code (OCP Violation)

```java
class Discount {
    double calculate(String type){
        if(type.equals("Student")) return 10;
        if(type.equals("Festival")) return 20;
        return 0;
    }
}
```

✅ SOLUTION (Apply OCP)

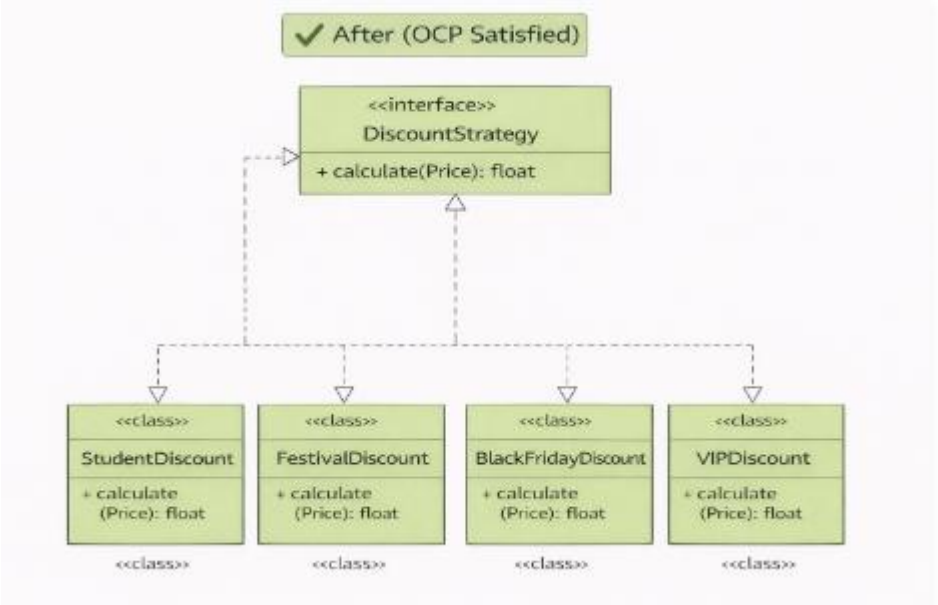
```java
interface Discount {
    double calculate();
}

class StudentDiscount implements Discount {
    public double calculate(){ return 10; }
}

class FestivalDiscount implements Discount {
    public double calculate(){ return 20; }
}
```

Explanation

Now when a new discount comes:
➡ Create a new class
➡ No modification to old code
➡ No risk to stability

System becomes future-proof.

UML Diagram



3. Liskov Substitution Principle (LSP)

Definition

Subclasses must work in place of their parent class without errors.

Scenario: Vehicle Management System

There is a company called **"GoFast"**. They provided all kinds of transportation services: taxis, buses, etc. The CEO hired a team of developers to create a **Vehicle Management System**.

Step 1: The Base Vehicle

The developers first created a base class called Vehicle. This class had a method drive() because every vehicle in the world can "drive" (or move).

```
class Vehicle {
    public void drive() {
        System.out.println("Vehicle is driving");
    }
}
```

At this point, everything seems fine. Taxi, Bus, and Car could inherit from this class:

```
java

class Car extends Vehicle { }

class Bus extends Vehicle { }
```

All cars and buses **work perfectly** with the Vehicle system. The company can call drive() on any Vehicle, and it behaves correctly.

Problem Over Time

Now, the company decides to deliver packages using **Delivery Drones**. But drones **don't drive**, they **fly**. The developer, trying to reuse the Vehicle class, writes:

```
java

class DeliveryDrone extends Vehicle {
    @Override
    public void drive() {
        throw new UnsupportedOperationException("Drone cannot drive!");
    }
}
```

At first, this seems okay because "we don't use drones like normal vehicles." But soon, a problem arises.

```
java

Vehicle v = new DeliveryDrone();
v.drive();   // Crash!
```

The system crashes because DeliveryDrone **violated the expectation set by** Vehicle. Any code expecting a Vehicle to "drive" should work with **all vehicles**, but now the drone breaks it.

This is exactly what **Liskov Substitution Principle** warns against..

✅ SOLUTION (Apply LSP)

The developers decides to redesign. They notice that **not all vehicles drive**. So they create **interfaces** or separate base classes:

```java
interface Vehicle {
}
```

(No methods — just a general type)

```java
interface DriveableVehicle {
    void drive();
}

interface FlyableVehicle {
    void fly();
}
```

```
class Car implements DriveableVehicle {
    public void drive() {
        System.out.println("Car driving");
    }
}

class Bus implements DriveableVehicle {
    public void drive() {
        System.out.println("Bus driving");
    }
}

class DeliveryDrone implements FlyableVehicle {
    public void fly() {
        System.out.println("Drone flying");
    }
}
```

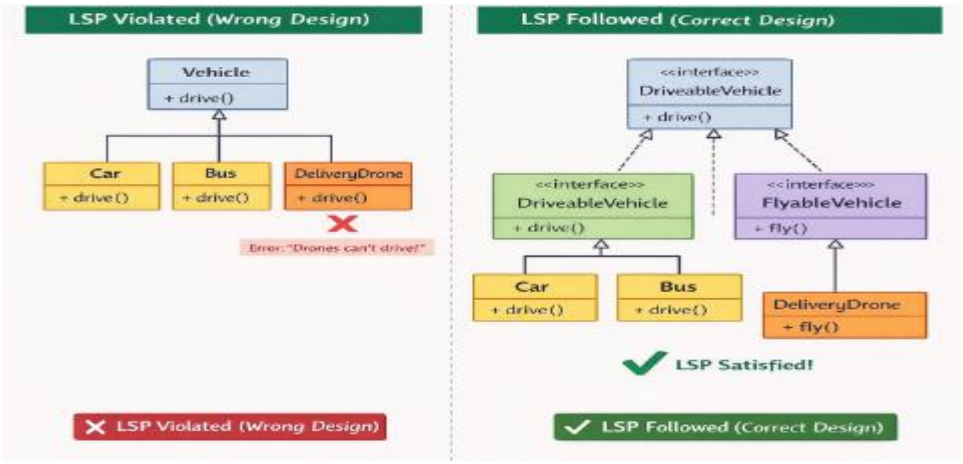
Explanation

Now, Car and Bus inherit from DriveableVehicle, and Drone inherits from FlyableVehicle:

Now, every subclass **fits the expectations of its parent**, and LSP is satisfied.

- DriveableVehicle can always be "driven."
- FlyableVehicle can always "fly."

UML Diagram



4. Interface Segregation Principle (ISP)

Definition

Clients shouldn't be forced to depend on methods they do not use.

If an interface is **too big (fat)**, classes are forced to implement methods that **don't make sense** for them.According to the interface segregation principle, you should break down "fat" interfaces into more granular and specific ones. Clients should implement only those methods that they really need.

Scenario: A Library for Cloud Services

Imagine you created a library called **CloudLink**, which makes it easy for apps to connect to cloud providers.

- **At first**, it only supported **Amazon Cloud**, so you included **all possible features**: compute, storage, database, AI, analytics, messaging, IoT etc.
- You created **one big interface** called CloudProvider that looked like this:

```java
interface CloudProvider {
    void compute();
    void storage();
    void database();
    void ai();
    void iot();
}
```

```java
class AmazonCloud implements CloudProvider {
    public void compute() { }
    public void storage() { }
    public void database() { }
    public void ai() { }
    public void iot() { }
}
```

Works perfectly for Amazon Cloud, because Amazon supports all these features.

Problem Over Time

Later, you want to support **TinyCloud**, a new cloud provider.

- Problem: TinyCloud **doesn't have AI, IoT, or messaging**.
- If your class tries to implement CloudProvider, you're **forced to add stubs** (empty methods) for features TinyCloud doesn't support:

❌ Code (ISP Violation)

```
class TinyCloud implements CloudProvider {

    public void compute() {
        System.out.println("Compute supported");
    }

    public void storage() {
        System.out.println("Storage supported");
    }

    public void database() { } // Empty
    public void ai() { }       // Empty
    public void iot() { }      // Empty
}
```

- Ugly, error-prone, and confusing for users.
- This is exactly the **fat interface problem** that the Interface Segregation Principle warns against.

✅ SOLUTION (Apply ISP)

Break the Interface into smaller pieces

Instead of one giant CloudProvider interface, we create **smaller, specific interfaces**:

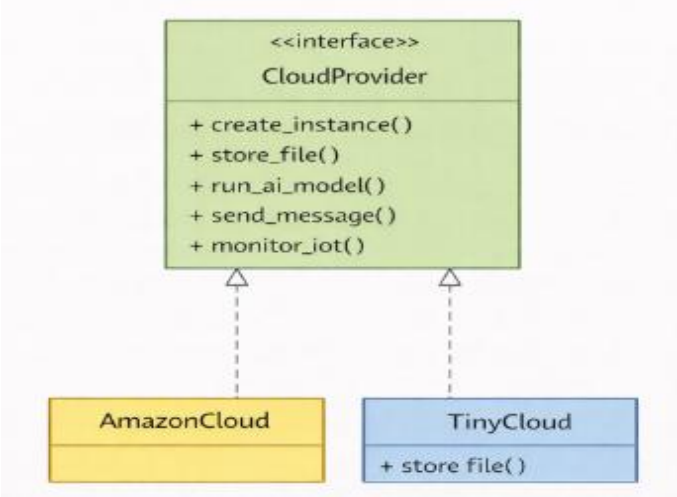
```
interface ComputeProvider {
    void compute();
}

interface StorageProvider {
    void storage();
}

interface DatabaseProvider {
    void database();
}

interface AIProvider {
    void ai();
}

interface IoTProvider {
    void iot();
}
```

```
class AmazonCloud implements ComputeProvider,
                             StorageProvider,
                             DatabaseProvider,
                             AIProvider,
                             IoTProvider {

    public void compute() { }
    public void storage() { }
    public void database() { }
    public void ai() { }
    public void iot() { }
}
```

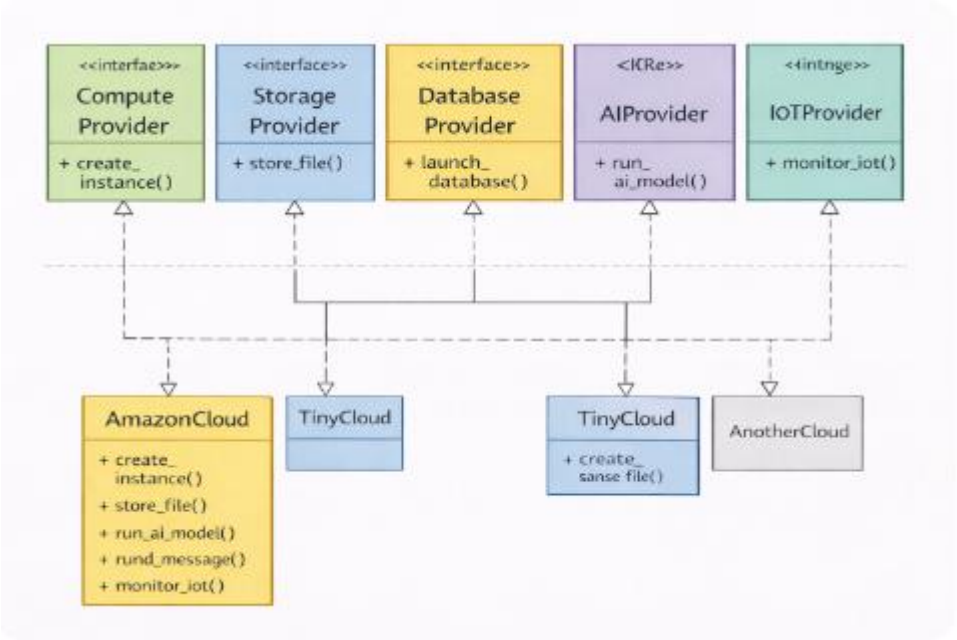
java

```
class TinyCloud implements ComputeProvider, StorageProvider {

    public void compute() {
        System.out.println("TinyCloud compute");
    }

    public void storage() {
        System.out.println("TinyCloud storage");
    }
}
```

Much cleaner! No empty methods, no confusion. Each cloud provider implements **only what makes sense**.



Before (ISP Violation):
One big CloudProvider interface forces all cloud services to implement methods they don't support, leading to empty or useless methods.



After (ISP Satisfied):
AmazonCloud

- Implements **all five interfaces** (compute, storage, database, AI, IoT)
- Because Amazon supports all features

TinyCloud

- Implements **only the interfaces it supports** (compute, storage)
- Ignores AI, IoT, database because TinyCloud doesn't support them

5. Dependency Inversion Principle (DIP)

Definition

High Level classes should not depend on low level classes. Both should depend on abstraction. Abstraction should not depend on details. Detail should depend on abstraction.

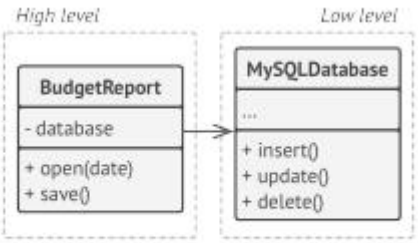
Scenario: The Budget Report and the Database

A company has a **BudgetReport** system that generates monthly financial reports.

They have a **high-level class**: BudgetReport → handles business logic.

They have a **low-level class**: MySQLDatabase → reads and writes data.

Initially, BudgetReport talks **directly to the database**:



BEFORE: a high-level class depends on a low-level class.

```
public class MySQLDatabase {

    public void insert(String data) {
        System.out.println("Insert into MySQL: " + data);
    }

    public void update(String data) {
        System.out.println("Update MySQL: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MySQL ID: " + id);
    }
}
```

```
public class BudgetReport {

    private MySQLDatabase database;

    public BudgetReport() {
        database = new MySQLDatabase(); // Direct dependency
    }

    public void saveReport(String report) {
        database.insert(report);
    }
}
```

Problem

Any change in Database (new database version, API changes) will **break BudgetReport**. High-level business logic **depends on low-level implementation details**.

This **violates DIP**.

✅ SOLUTION (Apply DIP)

Introduce Abstraction

Create an **interface** Database

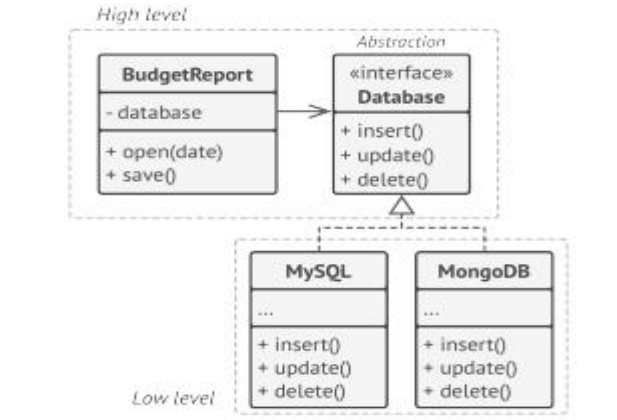
```
public interface Database {

    void insert(String data);
    void update(String data);
    void delete(int id);
}
```

```
public class MySQL implements Database {

    public void insert(String data) {
        System.out.println("Insert into MySQL: " + data);
    }

    public void update(String data) {
        System.out.println("Update MySQL: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MySQL ID: " + id);
    }
}
```

```
public class MongoDB implements Database {

    public void insert(String data) {
        System.out.println("Insert into MongoDB: " + data);
    }

    public void update(String data) {
        System.out.println("Update MongoDB: " + data);
    }

    public void delete(int id) {
        System.out.println("Delete from MongoDB ID: " + id);
    }
}
```

```
public class BudgetReport {

    private Database database;

    public BudgetReport(Database database) {
        this.database = database;
    }

    public void saveReport(String report) {
        database.insert(report);
    }
}
```

```
public class Main {
    public static void main(String[] args) {

        Database db = new MySQL(); // Can switch to MongoDB
        BudgetReport report = new BudgetReport(db);

        report.saveReport("January Budget");
    }
}
```



AFTER: low-level classes depend on a high-level abstraction.

Explanation

The **Database interface** contains common operations like:

- Read data
- Write data

These operations are basic functions that **any database can easily implement**, whether it is MySQL, MongoDB, or another database.

Because all databases support reading and writing data, they can implement this interface without difficulty.

The **BudgetReport class talks directly to the Database interface**, not to a specific database like MySQL or MongoDB.

This means:

- BudgetReport only knows about read and write operations
- It does not know which database is being used
- It does not depend on database details

EXERCISE

Task 1 — Applying SRP (Online Learning Platform Scenario)

An **online learning platform** manages:

- Student enrollment
- Course content delivery
- Progress tracking
- Email notifications to students

Currently, one class handles all these responsibilities.

Your Task

1. Explain why this violates the Single Responsibility Principle.
2. Identify at least three separate responsibilities.
3. Redesign the system by separating responsibilities into different classes.
4. Draw a UML class diagram of your improved design.
5. Provide sample code for your design.

Task 2 — Applying OCP (Movie Streaming Service Scenario)

A **movie streaming service** calculates subscription costs. Initially, it supports:

- Basic Plan
- Premium Plan

Later, the company introduces:

- Family Plan
- Student Plan
- Ad-supported Plan

The current pricing class must be edited every time a new plan is added.

Your Task

1. Explain why modifying the same class repeatedly is risky.
2. Redesign the system using OCP.
3. Draw a UML diagram showing how new plans can be added easily.
4. Write sample code supporting extensibility.

Task 3 — Applying LSP (Gaming System Scenario)

A **gaming system** has a base class GameCharacter with a method attack().

Some characters (like Warriors) can attack, but characters like **Healers** mainly restore health and do not attack.

Your Task

- Explain how forcing Healer to implement attack() violates LSP.
- Redesign the hierarchy properly.
- Draw UML diagrams before and after redesign.
- Provide corrected code.

Task 4 — Applying ISP (Smart Home System Scenario)

A **smart home system** controls:

- Smart Lights
- Smart Speakers
- Smart Door Locks

A large interface includes:

- playMusic()
- adjustBrightness()
- lockDoor()

Not all devices use all methods.

Your Task

1. Explain why this violates ISP.
2. Split the interface into smaller ones.
3. Draw UML diagrams.
4. Provide example implementation.

Task 5 — Applying DIP (E-Library System Scenario)

An **e-library system** stores books using a local database. Later, the library wants to support:

- Cloud storage
- External digital archives

The system directly depends on LocalDatabase class.

Your Task

1. Explain why direct dependency is problematic.
2. Redesign using DIP.
3. Draw UML showing abstraction usage.
4. Provide sample flexible code.

Task 6 — Mini Case Study (Integrated SOLID)

Consider a **Fitness Tracking App** that manages:

- Workout tracking
- Diet plans
- Notifications
- User progress reports

Your Task

1. Identify where at least three SOLID principles apply.
2. Suggest design improvements.
3. Draw UML diagram.
4. Provide sample code for one improvement.